

Incident Response Plan

Security @ Open Source w/ SLU
Spring 2026

Purpose

When something goes wrong, nobody should be guessing what to do. This plan takes the guesswork out of incident response so the team can focus on fixing things instead of figuring out where to start.

This document is meant to outlast the people who wrote it. Future security teams should not have to rebuild from scratch. They should be able to pick this up, understand what we built, and go from there.

Mission Statement

We support open science and sustainable innovation by delivering value for researchers with software and tools and providing students with realistic software development experience.

Security Priorities

When responsibilities conflict or resources are limited, these are the values weighed in order:

1. **Open source compliance.** We operate as an open source organization. Actions that would compromise that status are not on the table.
2. **Integrity of our software to our clients.** What we ship should be trustworthy. Our clients rely on it.
3. **Confidentiality and privacy of our contributors.** The people who build with us should not have to worry about their information being exposed.

This order is not a license to ignore lower priorities. It is a tiebreaker for when hard calls need to be made quickly.

Scope

This plan covers all active repositories and systems under Open Source w/ SLU's control. Priority is tiered as follows:

- Active projects in development are the top priority.
- Inactive projects are lower priority, with one exception: the `oss.slu.website` is front-facing and carries higher visibility even as a static site.
- Mission critical projects have their own section and carry elevated response expectations regardless of their position in the normal priority queue.

Roles and Responsibilities

- **Security team lead:** owns the response, makes escalation calls, serves as the primary point of contact for SLU ITS.
- **Security contributors:** triage alerts, investigate, and patch under lead direction.
- **Tech leads for affected projects:** must be looped in for Code Scanning alerts before any patch is applied. They own their codebase.
- **SLU ITS:** escalation contact for incidents involving enterprise-level assets or requiring action beyond what the security team can take unilaterally.

Alert Types, Severity, and Response Times

Three main alert categories are managed through GitHub, plus externally submitted advisories. Severity and expected response time vary by type.

Dependabot

Triggered when a dependency has a known vulnerability. The fix is typically a version bump. Still needs review before applying.

- **Expected response:** Review within one week.

Code Scanning

The AI scanner flags code patterns that match known vulnerability signatures. These are not definitive findings and need investigation before any fix is applied. Copilot can generate an automatic patch, but this must be reviewed with the tech lead for the affected project first.

- **Expected response:** Triage within 1–2 business days.

Secret Scanning

Triggered when something resembling a secret (API key, token, password, private key) appears in a repository. These are almost always real problems. Treat them as serious until proven otherwise.

- **Expected response:** Acknowledge immediately. Rotate or revoke the exposed credential within the same day.

External Advisories

If a security researcher or someone from the broader security community submits a vulnerability report through our repository, that becomes our responsibility to triage and respond to. The relevant tech lead should be brought in. Depending on severity, this may warrant pausing other roadmap work.

- **Expected response:** Acknowledge receipt within 24 hours.

Incident Response Lifecycle

The phases of a response, in order.

Phase 1: Detection

How the incident or alert surfaces. Covers:

- Which systems or channels generate alerts (GitHub, SLU ITS notification, external submission)
- Who is responsible for monitoring each channel
- What constitutes a confirmed detection vs. a false positive

Phase 2: Triage

How we determine what we are dealing with before committing to a response path. Covers:

- Severity classification by alert type (see Alert Types section)
- Assigning ownership for the response
- Decision point: does this warrant pausing roadmap work (see Pause Conditions)
- Logging the triage finding in the incident log

Phase 3: Containment

Immediate steps to limit the blast radius before the full investigation is complete. Covers:

- Containment actions by alert type (e.g., revoking a leaked credential, restricting repository access)
- Who authorizes containment steps
- Internal status communication during containment

Phase 4: Investigation

Determining what happened, what was affected, and how far it reached. Covers:

- Tools and methods used (GitHub audit log, repository history, access records)
- Scope assessment: what data or systems were exposed
- Ongoing documentation requirements during the investigation
- When to escalate to SLU ITS or other parties based on findings

Phase 5: Eradication

Removing the vulnerability or threat from the environment. Covers:

- Patch and remediation steps by alert type
- Verification that the threat has been fully removed
- Coordination with tech leads for codebase-level fixes

Phase 6: Recovery

Restoring normal operations and confirming nothing is still broken. Covers:

- Steps to restore affected systems or repositories to a known good state
- Validation and testing before returning to normal operations
- Who signs off that recovery is complete

Phase 7: Post-Incident Review

What we learned, what worked, what did not, and what changes to make. Covers:

- Who leads the review and who participates
- What gets documented: timeline, decisions made, gaps identified
- How findings feed back into this document and other security documentation

This phase is not optional. A review that never gets written might as well not have happened.

Incident Log

Every incident must be logged at the time it occurs, not after the fact. The log entry should include:

- **Date and time detected:** when the alert first surfaced or the incident was identified.
- **Reported by:** who flagged or discovered the incident.
- **Affected repository or system:** the specific repo, service, or asset involved.
- **Alert type:** Dependabot, Code Scanning, Secret Scanning, or External Advisory.
- **Severity classification:** as determined during triage.
- **Evidence:** screenshots, log excerpts, audit trail links, or any other artifacts captured at time of detection.
- **Containment steps taken:** actions taken immediately to limit exposure.
- **Remediation steps:** what was patched, revoked, updated, or removed and when.
- **Final resolution:** confirmation that the threat was eradicated and the system returned to normal.
- **Post-incident review date:** when the review was completed and by whom.

Logs are retained as part of the organization's security record. Future teams should be able to reconstruct any incident from the log entry alone.

Escalation Path

Not everything stays internal. The following defines how escalation works and what happens to communication during an incident.

When to Escalate

- The security team handles alerts and incidents that can be contained and resolved within the scope of project repositories.
- Escalate to SLU ITS when the incident involves SAML configuration, enterprise GitHub settings, compromised SLU-managed credentials, or any asset under ITS administrative control.
- Escalate immediately for Secret Scanning alerts involving tokens with active enterprise access.

Points of Contact

- Primary SLU ITS contact: *[to be filled in]*
- Secondary or off-hours contact: *[to be filled in]*
- What to have ready when escalating: affected systems, timeline of detection and containment

steps taken, any evidence collected.

Communication During an Incident

- Internal communication happens between the security team lead, the affected tech lead, and SLU ITS if escalated. No broader communication until the scope is understood.
- Only the security team lead is authorized to communicate outward about an active incident, whether to faculty, SLU ITS, or external parties.
- Project tech leads are informed when their repository or system is directly affected. They are not informed of unrelated incidents.
- Faculty and program leadership are notified at the security team lead's discretion, typically after containment and when the scope is clear.
- What gets documented during an incident: all decisions made, who was contacted and when, what information was shared, and any commitments made to external parties. Log it in real time.

Pause Conditions

Some incidents require pausing roadmap work entirely. The following situations likely warrant it:

- A serious external advisory requires a coordinated response
- A Secret Scanning alert points to a leaked credential with active access
- SLU ITS requests immediate action on something affecting the enterprise
- A vulnerability is found in a mission critical project prior to or during its SLU ITS review

The security team lead makes the call. Document it when it happens.

Project-Specific Notes

oss_slu_website

Front-facing, even though it is a static site. Lower inherent attack surface, but public-facing means higher visibility. Treat it accordingly.

Mission Critical Projects

Certain projects carry strategic importance to the organization or the university. These may involve sensitive integrations (Okta, RBAC), external review by SLU ITS, or direct impact on university operations. A Threat Model is required for any project in this category. Security concerns flagged during SLU ITS review are treated as high priority regardless of what else is on the roadmap.

GitHub SAML / Okta SSO

SAML authentication is required across the organization. Any incident involving authentication or account access must account for how SAML affects internal and external contributors differently. External contributors are not covered by SAML the same way internal members are, and that distinction matters during an investigation.

Ongoing Alert Management

Alert clearance is a continuous responsibility, not a break-glass activity. The target is under five open alerts per active repository. Most Dependabot alerts are straightforward patches and should not be left to pile up.

A growing backlog is a real risk. Stay on top of it.

Document Maintenance

This document is reviewed at the start of each semester and after any significant incident. Whoever owns it is responsible for making sure the contact information, role assignments, and escalation paths reflect the current team. If something changes mid-semester that affects this document, update it then.